

Virtual Environments

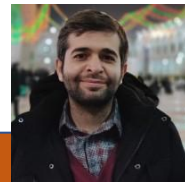
Using venv and conda

Reproducibility

extent to which consistent results are obtained when an experiment is repeated.



Psst..
it runs perfectly on **my** machine!

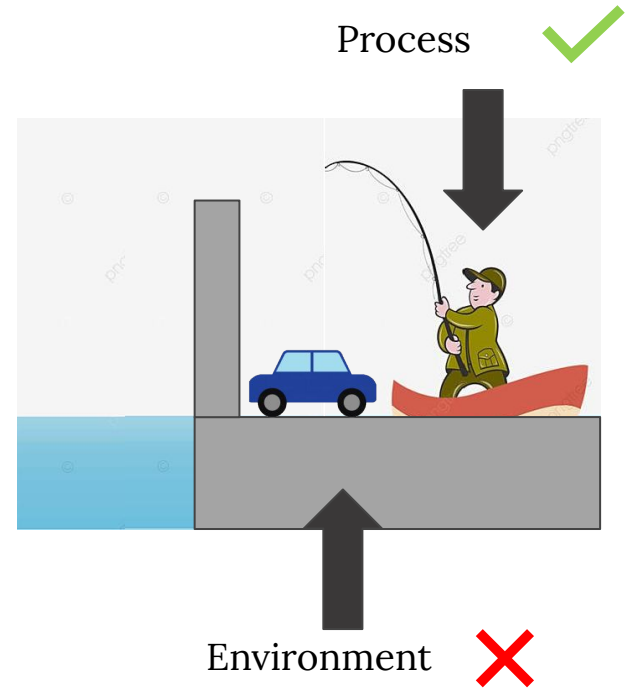
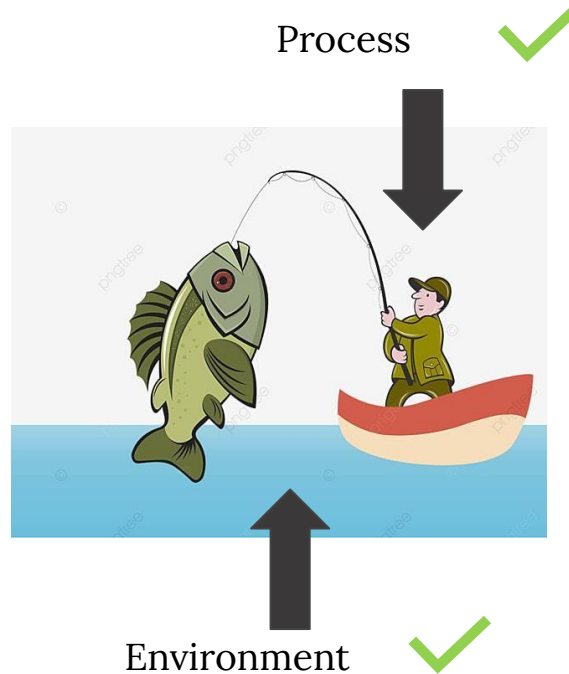


Yo, man..
Your code is

Yo...for real??
But which

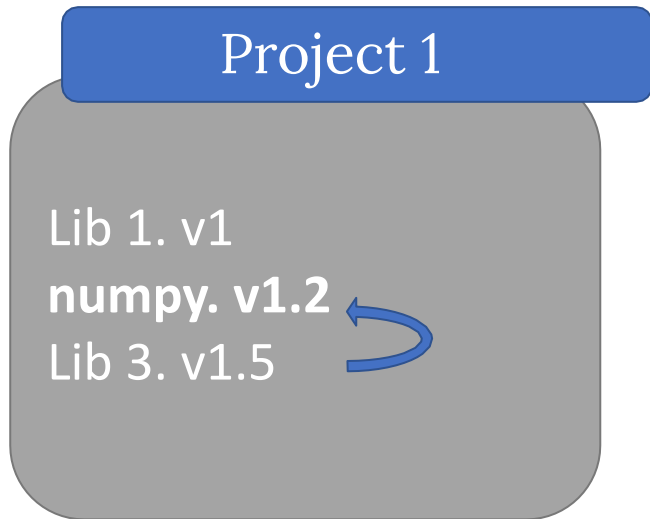
Reproducibility

- **Process**
Makefile
- **Environment**
??



Why Virtual Environments?

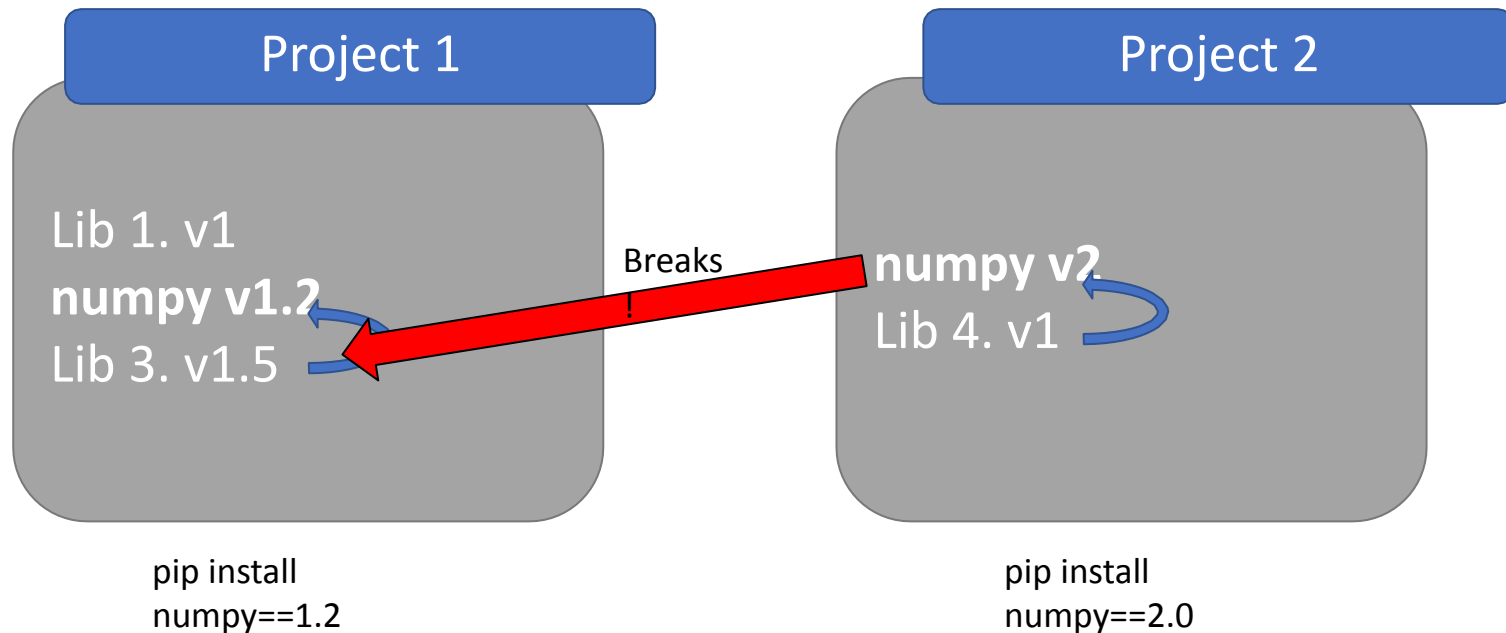
ISSUE 1: having control on the **actual project dependencies**
(and sub-dependencies)



```
pip install  
numpy==1.2
```

Why Virtual Environments?

ISSUE 2: Allowing projects with conflicting dependencies



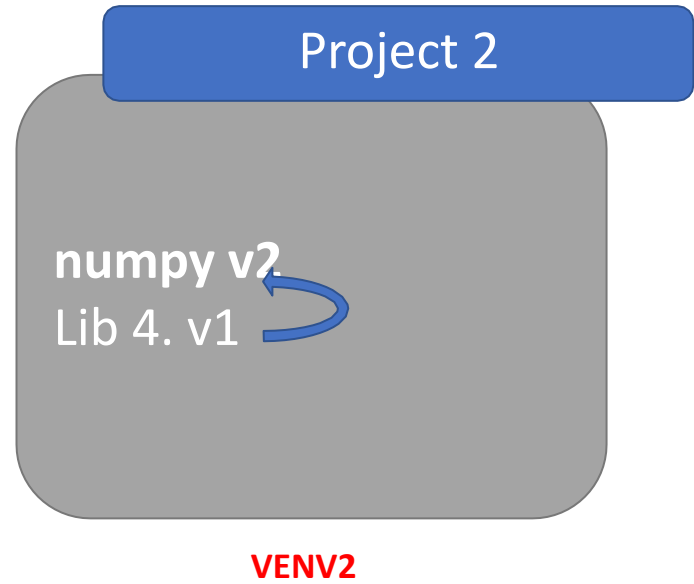
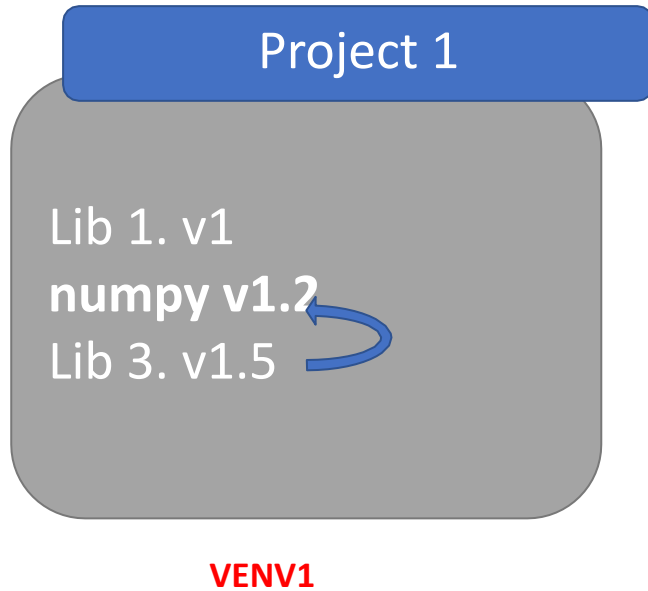
Why Virtual Environments?

- Installing packages globally causes conflicts.
- One project might break another by upgrading a package.
- Hard to reproduce the setup on another machine.
- Virtual environments make sure it runs the same way everywhere.

What Is a Virtual Environment?

- A self-contained Python setup for your project.
- Includes its own Python interpreter and packages.
- Independent of the global Python installation.

Virtual Environment



Tools for Virtual Environments

- venv – Built-in, uses pip, supported on all OS.

```
> pip install virtualenv
```

- conda – From Anaconda/Miniconda, uses conda & pip.

Creating a venv

Create environment

```
> python -m venv myenv
```

Activate

```
(Windows) myenv\Scripts\activate
```

```
(Linux/Mac) source myenv/bin/activate
```

Deactivate

```
> ./deactivate
```

Creating a Conda Environment

Create environment with specific Python version

```
> conda create --name myenv python=3.10
```

Activate

```
> conda activate myenv
```

Deactivate

```
> conda deactivate
```

Installing Packages

- With pip (in venv or conda):

```
> pip install numpy pandas
```
- With conda (only in conda envs):

```
> conda install numpy pandas
```
- You can use both in conda environments.

requirements.txt

Save environment packages

```
> pip freeze > requirements.txt
```

Reinstall later

```
> pip install -r requirements.txt
```

- Useful for sharing with others

Using Virtual Envs in Jupyter

Inside the environment:

```
> pip install ipykernel
```

Add to Jupyter kernels

```
> python -m ipykernel install --user  
--name=myenv --display-name "Python  
(myenv) "
```

- Then select "Python (myenv)" in Jupyter

Sharing Your Project

1. Copy the environment folder (only if OS is the same)
2. Use requirements.txt and recreate the environment
3. Use Docker – works everywhere